2024 4th International Symposium on Artificial Intelligence and Intelligent Manufacturing (AIIM) | 979-8-3315-4172-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/AIIM64537.2024.10934357

Research on the Remaining Useful Life Prediction Model of Complex Equipment

Lei Tang
Science and Technology on Reactor System Design Technology Laboratory
Nuclear Power Institute of China
Chengdu, China

Junhao Zhang
Glasgow College
University of Electronic Science and Technology of China
Chengdu, China

Yifei Tang
School of Information and Software Engineering
University of Electronic Science and Technology of China
Chengdu, China

Mengjuan Liu
School of Information and Software Engineering
University of Electronic Science and Technology of China
Chengdu, China
*Corresponding author: mjliu@uestc.edu.cn

***Abstract*—With the rapid advancement of artificial intelligence technology, the development of intelligent manufacturing has become an inevitable trend. Utilizing AI technology to ensure the safe operation of factories is a crucial aspect of smart factories. In particular, research focused on fault diagnosis and remaining useful life (RUL) prediction models based on machine learning has garnered significant attention from both academia and industry. This paper explores RUL prediction models for complex equipment, introduces four representative models, and analyzes their performance using two benchmark datasets, C-MAPSS and N-CMAPSS. Finally, the paper discusses the future research and application trends of artificial intelligence technology in complex equipment operation and maintenance.**

Keywords-remaining useful life prediction; machine learning; intelligent manufacturing

I. INTRODUCTION

With the rapid development of technologies such as artificial intelligence (AI) and the industrial Internet of Things (IIoT), intelligent manufacturing has emerged as a significant trend in industrial advancement. Intelligent manufacturing seeks to integrate industry expertise with vast amounts of industrial data by utilizing artificial intelligence to enhance production and decision-making processes, improve efficiency, and reduce costs. Research and application of intelligent manufacturing are experiencing robust growth and have been implemented in the automobile and aircraft manufacturing sectors. This paper specifically focuses on the use of AI technology in prognostics and health management (PHM) of complex equipment, particularly in predicting RUL.

Equipment failures due to aging during long-term operations cannot be entirely avoided. If these failures are not addressed effectively, they can lead to production interruptions and economic losses, and in the worst cases, they may result in casualties and public safety hazards [1]. Artificial intelligence-based health management involves embedding sensors in complex equipment to monitor its health continuously [2]. By collecting real-time status parameters, the PHM system can provide early warnings at the initial stages of equipment degradation. It allows operation and maintenance personnel to develop timely maintenance plans and operational strategies.

This paper focuses on the RUL prediction task in complex equipment health management. Specifically, it aims to use time-series data from equipment operations to forecast how much longer the equipment can run before failure [3]. Traditional RUL prediction methods, which are based on physical or statistical models, require researchers to possess specialized domain knowledge and often struggle to model complex systems effectively. Currently, mainstream data-driven approaches primarily employ machine learning to analyze the relationships between equipment state parameters and the RUL, significantly lessening the reliance on domain experts. Additionally, the machine learning models utilized for this kind of modeling have evolved from earlier, simpler structures like logistic regression (LR) and support vector machines (SVM) to more recent deep learning architectures such as convolutional neural networks (CNN) and recurrent neural networks (RNN).

The main contributions of this paper are as follows:

- We present a general implementation framework for predicting RUL of complex equipment.
- We implement four representative RUL prediction models and quantitatively compare their performance across two benchmark datasets.
- We provide the source code for these RUL prediction models on GitHub, which will assist researchers in related fields in quickly conducting RUL modeling for various scenarios.

II. RUL PREDICTION MODELS

RUL prediction is a key area of research in PHM. Currently, the predominant approaches to RUL prediction utilize supervised machine learning methods. This section begins by introducing the RUL prediction task, followed by a general modeling framework, and concludes with a description of four representative models for RUL prediction.

Science and Technology Programs of Sichuan Province of China (No. 2023YFH0082) and NSFC (No. 62372083)

A. RUL Prediction Task

For RUL prediction of complex equipment, we hope to predict the time that the current equipment can continue to operate until complete failure through the changing trend of equipment operating state parameters. The RUL prediction function is denoted as $\hat{y} = f(\boldsymbol{x})$, where $\boldsymbol{x}$ represents the time series of state parameters, and $\hat{y}$ represents the predicted time from the current moment to complete failure. In supervised machine learning modeling, we need to optimize the parameters of the function $f(\cdot)$, that is, the model parameters, through sample learning. The model can directly output $\hat{y}$ according to a given state parameter sequence $\boldsymbol{x}$.

B. General Modeling Framework

To create a RUL prediction model using supervised machine learning methods, it is essential to design the input sample format, the structure of the RUL prediction model, and the loss function. Fig. 1 illustrates the framework we developed for RUL modeling. Whenever dealing with an RUL prediction task for complex systems, the RUL prediction model can be established by following the steps outlined in this framework.

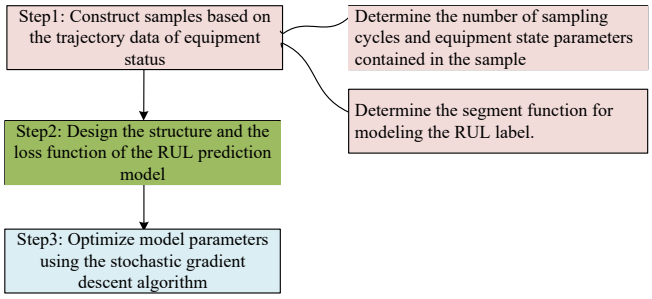

Figure 1. Modeling Framework.

To optimize model parameters, we first construct a training sample set based on trajectory data from equipment operation. This trajectory data must encompass the full range of operating states, from healthy operation to complete failure. For each sample, we need to determine the number of sampling cycles and the state parameters it includes, as well as the RUL label (time elapsed since the last sampling cycle until the equipment fails). One of the key challenges is determining the RUL label, as it is not possible to infer RUL from operating parameters when the equipment is in a healthy state. A practical approach is to use a piecewise linear function to separate the RUL into two sections. The healthy state can be represented by a constant value, while a linearly decreasing time function is used to compute the RUL label when the equipment in degradation state.

Next, we need to design the structure of the RUL prediction model. We can choose from several options, including a simple SVM [4], a fully connected feedforward neural network [5], a convolutional neural network [6], a recurrent neural network [7], or the latest Transformer architecture [8]. Additionally, we need to determine the loss function for training. Since this is a prediction task, the absolute error is typically chosen as the loss.

Finally, we apply the stochastic gradient descent algorithm to optimize the model parameters until the loss converges or we reach the maximum number of training rounds. Once this process is complete, we can select the optimal model parameters for the RUL predictor.

C. Representative Models

1) FNN-based Structure: The feedforward neural network (FNN) is the earliest neural network structure. The FNN is used to construct the RUL prediction model, as shown in Fig. 2. Here, the input of the feedforward neural network is a one-dimensional vector. The number of hidden layers and the number of neurons in each layer are adjustable hyper-parameters. Each neuron uses ReLU as the activation function. Finally, the output neuron outputs the RUL prediction value.

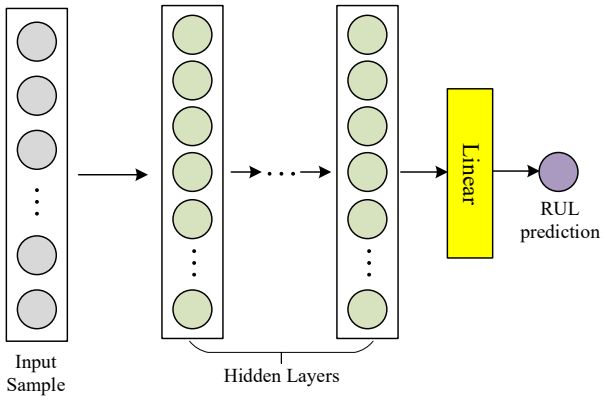

Figure 2. FNN Structure.

2) CNN-based Structure: Convolutional neural networks can mine the implicit information between features and have achieved excellent results in the field of images. Establishing an RUL prediction model based on CNNs can be an excellent choice. The basic idea is to construct a two-dimensional matrix from multiple operating state parameters of continuous time steps and use the convolution operation to extract the key information of this two-dimensional matrix. Many RUL prediction models based on CNN, and the representative one is TCN [9]. Its structure is shown in Fig. 3. The basic idea is to regard the input two-dimensional matrix as a whole and extract the implicit information between different features at different time steps in the sample through convolution operations. The dimension of each convolution kernel is $W \times N$, where N is the number of features contained in the sample, and W is the number of time cycles that need to be mined by the convolution operation. Finally, several feature maps are spliced into a one-dimensional vector and input into a hidden layer. The RUL prediction value is calculated through the final output neuron. Compared with FNN, TCN can better mine the implicit information of different features between different time steps in the sample and produce better prediction performance than FNN.

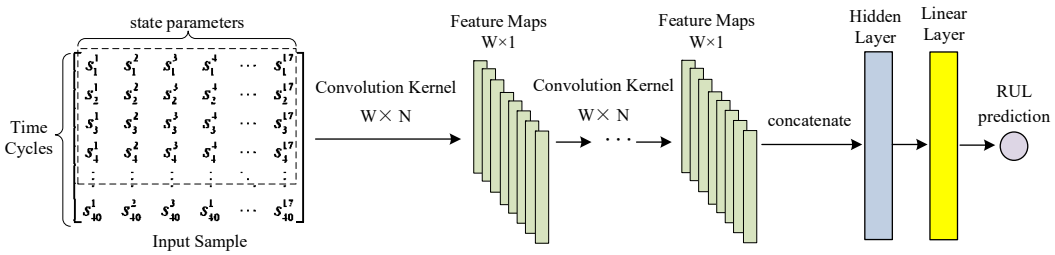

Figure 3. TCN Structure.

1) RNN-based Structure: CNN itself does not have memory functions. When extracting information from the sample, it does not consider the time relationship between state parameters at different time cycles. Therefore, RNNs are introduced into RUL prediction modeling to better complete the information mining of time series data. The representative model based on RNN is to construct an RUL prediction model by using the LSTM model, which is a variant of RNN, as shown in Fig. 4. Compared with FNN, the LSTM-based model does improve prediction performance. However, compared with TCN, LSTM only achieves better metrics on some datasets. This is because LSTM is not as capable as CNN in mining implicit information between features.

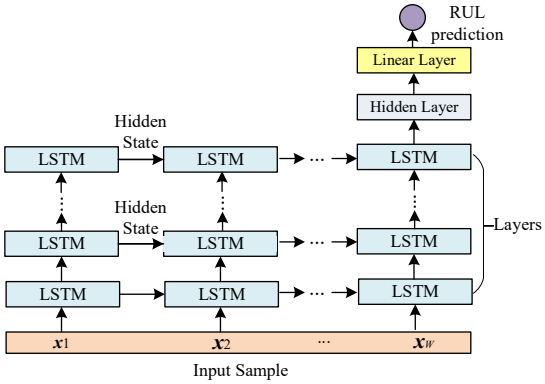

Figure 4. LSTM-based Structure.

2) Transformer-based Structure: In recent years, the self-attention mechanism has received extensive attention. The Transformer structure has been applied in natural language and image processing. The self-attention mechanism can perform similarly to complex neural networks with a simple structure. We have also implemented an RUL prediction model using the Transformer structure, as shown in Fig. 5. In the Transformer-based model, the state parameters of the time series are regarded as the feature vectors of the position sequence so that the model can capture the information between time series and features. In experiments, we observe that the Transformer-based scheme can obtain prediction performance similar to that of TCN.

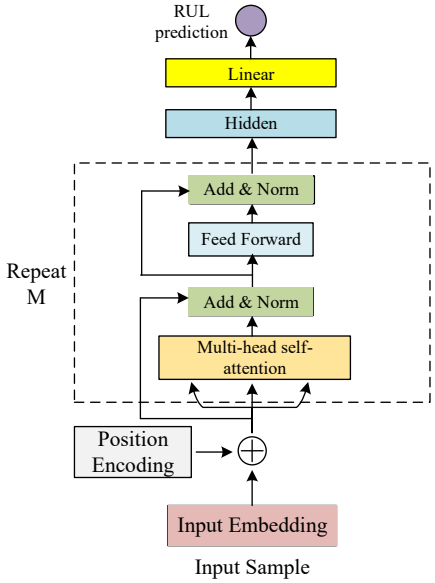

Figure 5. Transformer-based Structure.

II. Experiments and Results Analysis

A. Datasets

We evaluate the performance of four representative models using two aeroengine datasets. The C-MAPSS dataset, released by NASA in 2009, is designed for aero-engine RUL prediction tasks and has become a benchmark dataset. Based on varying working conditions and fault scenarios, this dataset is divided into four subsets—FD001 to FD004. Each subset offers sensor data trajectories from multiple aero-engine engines, capturing their operation from normal functioning to failure. To establish training and test samples, we segment the trajectory data according to the method described in Section II. For a detailed introduction to the C-MAPSS dataset, please refer to [10].

The N-CMAPSS dataset, a newer dataset released by NASA in 2021, serves not only for evaluating RUL prediction models but also for assessing fault classification models. It consists of eight subsets—DS01 to DS08. Our experiments present results only for two subsets (DS01 and DS03) due to space constraints. This dataset is simulated based on real flight data, leading to a substantial amount of sensor log data recorded in seconds. For

more information on the N-CMAPSS dataset, please refer to [11]. We preprocess the data by segmenting each flight into two parts to reduce computational costs. The mean values of the state parameters for the first and second halves of each flight are calculated. As a result, we obtain two time-step state parameter vectors for each flight, with their corresponding RUL label values identical. For the samples in the new dataset, the RUL label value indicates the remaining number of flights before the engine takes off.

B. Experimental Settings

Both datasets require data preprocessing, as well as the construction of training and testing samples. The detailed preprocessing steps can be referenced in [12]. The evaluation metrics used are RMSE and Score, as outlined in formulas (1) and (2). When the predicted RUL exceeds the actual RUL label, it can lead to more severe consequences than when the predicted RUL value is less than the actual RUL label. Consequently, for the Score, the penalty applied for overestimating will be greater than for underestimating. Smaller values are considered favorable for both metrics.

$$Score = \begin{cases} \sum_{i=1}^{N} e^{-\frac{d}{13}} - 1, if\ d < 0 \\ \sum_{i=1}^{N} e^{-\frac{d}{10}} - 1, if\ d \geq 0 \end{cases}, \text{where } d = \hat{y} - y \quad (1)$$

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N} d_i^2}, \text{where } d = \hat{y} - y \quad (2)$$

TABLE I. HYPERPARAMETER TUNING OF THE FNN-BASED MODEL

Metrics	Hidden Layers	FD001			FD002			FD003			FD004		
		Neurons in each Layer			Neurons in each Layer			Neurons in each Layer			Neurons in each Layer		
		50	100	150	50	100	150	50	100	150	50	100	150
RMSE	1	17.39	**16.27**	17.22	**14.74**	15.27	15.64	**15.89**	17.66	16.63	**16.85**	17.79	18.19
Score		522.78	**568.24**	649.32	**861.16**	992.30	2097.12	**483.94**	765.99	609.75	**1321.04**	1734.74	1905.70
RMSE	2	17.81	16.78	17.54	16.81	16.75	16.29	19.12	17.69	19.08	18.65	19.38	19.29
Score		838.38	643.32	892.77	1386.43	1722.16	1161.90	1099.53	881.77	1244.03	2292.28	2532.18	2637.88
RMSE	3	19.29	19.14	19.37	17.26	16.95	17.01	19.04	18.99	20.22	20.08	19.98	19.02
Score		997.42	1140.98	1162.96	1942.38	1515.94	1491.42	1172.58	1142.88	1510.47	4279.82	6253.74	2340.54

TABLE II. HYPERPARAMETER TUNING OF THE CNN-BASED MODEL

Metrics	Convolution Layers	FD001			FD002			FD003			FD004		
		Convolution kernels			Convolution kernels			Convolution kernels			Convolution kernels		
		4	6	8	4	6	8	4	6	8	4	6	8
RMSE	3	15.36	**13.79**	15.37	14.37	16.10	16.21	**13.91**	14.32	14.45	15.47	14.98	15.49
Score		452.06	**388.11**	428.79	974.58	1205.31	1289.23	**340.65**	361.21	420.88	2435.28	1029.21	1308.41
RMSE	4	15.43	15.79	15.20	**14.21**	15.10	14.49	14.49	15.04	14.26	15.62	**14.71**	14.66
Score		465.27	583.32	470.27	**809.04**	2045.85	1219.97	428.68	730.78	339.92	1216.57	**977.96**	1076.06
RMSE	5	15.16	15.32	19.53	15.10	14.56	14.84	13.99	14.63	23.77	14.72	14.63	16.46
Score		412.37	413.89	1003.82	1044.37	1036.42	1085.82	377.03	453.31	1472.03	1115.31	1095.54	1921.3

TABLE III. HYPERPARAMETER TUNING OF THE LSTM-BASED MODEL

Metrics	LSTM Layers	FD001			FD002			FD003			FD004		
		Hidden state dimension			Hidden state dimension			Hidden state dimension			Hidden state dimension		
		50	100	150	50	100	150	50	100	150	50	100	150
RMSE	1	15.93	**15.45**	17.58	**14.81**	15.39	16.34	14.58	15.49	**13.83**	17.87	16.02	15.70
Score		498.00	**465.54**	753.36	**1607.35**	1472.43	1537.93	386.19	473.65	**349.60**	3445.22	1465.11	1964.68
RMSE	2	16.18	17.56	15.76	17.34	17.99	17.71	15.74	15.14	15.54	**15.18**	16.83	17.79
Score		461.09	604.49	401.34	2056.14	2237.34	1920.39	641.19	667.09	538.08	**1578.47**	1771.75	2269.03
RMSE	3	16.42	17.09	17.03	17.50	17.34	18.27	15.95	15.07	14.92	18.24	16.84	17.04
Score		550.61	597.07	632.21	2040.12	2254.61	2454.87	938.25	754.72	481.56	2177.06	1674.31	1741.49

TABLE IV. HYPERPARAMETER TUNING OF THE TRANSFORMER-BASED MODEL

Metrics	Encoders	FD001			FD002			FD003			FD004		
		Attention Heads			Attention Heads			Attention Heads			Attention Heads		
		2	4	6	2	4	6	2	4	6	2	4	6
RMSE	3	**13.68**	15.44	14.94	15.79	15.65	14.80	**13.39**	15.65	13.44	16.59	17.17	17.53
Score		**387.47**	504.41	470.42	1364.04	1454.53	1452.58	**380.82**	568.53	340.38	1796.89	2436.52	2215.32
RMSE	4	13.74	15.45	14.80	15.89	16.11	15.38	15.30	14.99	15.56	16.42	16.40	16.56
Score		302.20	421.46	586.79	1312.72	1408.77	1511.01	458.27	471.67	583.49	2161.80	2405.48	2071.40
RMSE	5	14.14	14.84	15.37	15.53	15.21	**14.65**	14.21	15.34	15.28	16.53	**16.23**	16.74
Score		305.66	393.38	678.38	1623.64	1210.66	**1108.27**	441.38	838.47	411.08	1928.77	**1512.24**	1956.48

C. *Hyperparameter Tuning*

In industrial scenarios, since the number of samples is limited, the model structure's design may greatly impact performance. This subsection introduces the hyperparameter tuning of four representative models. Due to space limitations, we list only the parameter tuning results for the four subsets of C-MAPSS. The methods on N-CMAPSS are similar.

The FNN model has a simple structure. On the four subsets of C-MAPSS, we increase the number of hidden layers from 1 to 3, and the number of neurons in each layer is increased from 50 to 150. The experimental results are shown in Table I. It can be seen that when there is 1 hidden layer with 100 neurons, the performance of the model is optimal. On FD002-FD004, the model performance is optimal when the number of hidden layers is 1 and the number of hidden layer neurons is 50. On DS01 and DS03, since the number of training samples is very small, we set the number of neurons in each hidden layer to 20, 40, and 60, and the number of hidden layers is still increased layer by layer from 1 layer. The optimal performance on DS01 is with 2 hidden layers and 20 neurons in each layer. DS03 performs best on a structure with 1 hidden layer and 20 neurons in each layer.

For the TCN structure, we set the number of convolutional layers to 3, 4, and 5, respectively. On the four subsets of C-MAPSS, the convolution kernels used in each convolutional layer are 4, 6, and 8, respectively. On DS01 and DS03, the convolution kernels used in each convolutional layer are 3, 5, and 7. Table II lists the experimental results. On FD001, a structure with 3 convolutional layers and 6 convolution kernels in each layer can obtain the optimal performance. On FD002, a structure with 4 convolutional layers and 4 convolution kernels in each layer can obtain the optimal performance. On FD003, a structure with 3 convolutional layers and 4 convolution kernels in each layer is optimal. On FD004, a structure with 4 convolutional layers and 6 convolution kernels in each layer is optimal. On DS01 and DS03, a structure with 3 convolutional layers and 3 convolution kernels in each layer achieves the optimal performance.

For the LSTM structure, we set the number of LSTM layers to 1, 2, and 3, respectively. On the four subsets of C-MAPSS, the hidden state dimension of each layer is set to 50, 100, and 150. On DS01 and DS03, the hidden state vector dimension of each layer is 10, 20, and 30, respectively. Table III lists the results of hyperparameter tuning. On FD001, the LSTM model achieves the best performance when using a 1-layer LSTM network, and the hidden state vector dimension is 100. On FD002, when using 1 LSTM layer and the hidden state vector dimension is 50, the optimal performance is achieved. On FD003, the performance is optimal when using 1 LSTM layer, and the hidden state vector dimension is 150. For FD004, the performance is optimal when using 2 LSTM layers, and the hidden state vector dimension is 50. On DS01, the performance is optimal when using 1 LSTM layer, and the hidden state vector dimension is 10. While on DS03, the performance is optimal when using 1 LSTM layer, and the hidden state vector dimension is 20.

In the Transformer-based architecture, we configured the encoder modules to 3, 4, and 5, with the number of self-attention heads set to 2, 4, and 6, respectively. For the DS01 and DS03 datasets, the encoder modules were set to 2, 3, and 4, with the number of attention heads adjusted to 2, 3, and 4, respectively. The experimental results are presented in Table IV. For FD001 and FD003, the best performance is achieved with 3 encoder modules, each utilizing a 2-head self-attention mechanism. For FD002, the optimal setup involves 5 encoder modules, each with a 6-head self-attention mechanism. In the case of FD004, the configuration of 5 encoder modules, each with a 4-head self-attention mechanism, yields the best results. For DS01, the best performance is obtained using 2 encoder modules with a 3-head self-attention mechanism, while for DS03, the configuration of 3 encoder modules with a 2-head self-attention mechanism is the most effective.

The above results show that the structure of the RUL prediction model is different for different datasets (application scenarios), and there is no unified hyperparameter setting. When there are more samples to support a complex model structure, it will bring better performance. In the data set scenarios in this paper, the number of samples is not large, so the model structures are relatively simple.

D. *Experimental Results*

The evaluation metrics on the six data subsets are shown in Table V and Table VI. We can observe that the performance of FNN is the worst on three datasets. The overall performance is not as good as the other three models. LSTM is slightly better than FNN. TCN and Transformer lead in different datasets. TCN does not show an obvious defect for time series data, which shows that fully mining the combined information between features is also very important. Although LSTM considers the

information of state parameters on the time scale, it still has deficiencies in mining information between features. The performance of the Transformer is equivalent to that of TCN, indicating that the Transformer is a good choice in processing time series. In addition, in the case of limited samples, for different datasets, even for the same model, due to different hyperparameter settings, there is still a large difference in performance. Therefore, parameter tuning still needs to be performed for different datasets.

TABLE V. RMSE METRICS ON SIX DATASETS

Datasets	FNN	LSTM	TCN	Transformer
FD001	16.27	15.45	13.79	**13.68**
FD002	14.74	14.81	**14.21**	14.65
FD003	15.89	13.83	13.91	**13.39**
FD004	16.85	15.18	**14.71**	16.23
DS01	6.08	6.36	6.65	**5.52**
DS03	6.67	**5.79**	6.59	7.22

TABLE VI. SCORE METRICS ON SIX DATASETS

Datasets	FNN	LSTM	TCN	Transformer
FD001	568.24	465.54	388.11	**387.47**
FD002	861.16	1607.35	**809.04**	1108.27
FD003	483.94	349.60	**340.65**	380.82
FD004	1321.04	1578.47	**977.96**	1512.24
DS01	458.56	496.39	562.06	**394.74**
DS03	555.26	**450.79**	571.21	624.21

III. CONCLUSION

In this paper, we conduct an empirical study on the RUL prediction models for complex equipment. We implement four representative models and analyze their performance. With the advancement of AI and IIoT technologies, the application of AI in smart factories is becoming increasingly essential. However, unlike natural language or image data, the operational data from industrial equipment remains quite limited. This presents a significant challenge in training a universal PHM system. Additionally, ensuring production safety is crucial. Industry experts often prioritize the interpretability of these AI models when they are deployed in factories. Lastly, we anticipate future research trends in intelligent operation and maintenance.

First, in the realm of benchmark datasets, an increasing number of research institutions and companies are beginning to share their experimental designs, datasets, and code. This collaboration will lead to the creation of influential datasets and code repositories. More researchers will be encouraged to engage in intelligent operation and maintenance by making these shared resources available.

Second, in the realm of model design, more new models, such as graph neural networks, pre-trained models, and large models, will be introduced into the research of system state monitoring and fault diagnosis, greatly improving the prediction and diagnosis performance.

Third, in the realm of interpretability, there will be model interpretability tools or auxiliary analysis tools for specific equipment. These tools can provide explanations from aspects such as feature extraction, model architecture, and result evaluation, making the reasoning and decision-making process transparent and thereby promoting the implementation of research results in practical applications.

Finally, with the completion of various data centers, the practical application of data-driven state monitoring and fault diagnosis models in intelligent manufacturing is now feasible. In the future, research and systems will utilize real manufacturing operation data for training, verification, and deployment.

REFERENCES

[1] M. Xiong, H. Wang, Q. Fu, and Y. Xu, "Digital twin-driven aero-engine intelligent predictive maintenance," The International Journal of Advanced Manufacturing Technology, vol. 114, pp. 3751–3761, 2021.

[2] B. Rezaeianjouybari and Y. Shang, "Deep learning for prognostics and health management: State of the art, challenges, and opportunities," Measurement, vol. 163, Article 107929, 2020.

[3] C. Ferreira and G. Goncalves, "Remaining useful life prediction and challenges: A literature review on the use of machine learning methods," Journal of Manufacturing Systems, vol. 63, pp. 550–562, 2022.

[4] X. Wang, H. Gu, L. Xu, C. Hu, and H. Guo, "A SVR-based remaining life prediction for rolling element bearings," Journal of Failure Analysis and Prevention, vol. 15, pp. 548–554, 2015.

[5] Z. Tian, L. Wong, and N. Safaei, "A neural network approach for remaining useful life prediction utilizing both failure and suspension histories," Mechanical Systems and Signal Processing, vol. 24, no. 5, pp. 1542–1555, 2010, special Issue: Operational Modal Analysis.

[6] X. Li, Q. Ding, and J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," Reliability Engineering & System Safety, vol. 172, pp. 1–11, 2018.

[7] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," In 2017 IEEE International Conference on Prognostics and Health Management (ICPHM), pp. 88–95, 2017.

[8] Y. Mo, Q. Wu, X. Li, and B. Huang, "Remaining useful life estimation via transformer encoder enhanced by a gated convolutional unit," Journal of Intelligent Manufacturing, vol. 32, pp. 1997–2006, 2021.

[9] G. Sateesh Babu, P. Zhao, and X. Li, "Deep Convolutional Neural Network Based Regression Approach for Estimation of Remaining Useful Life," In Database Systems for Advanced Applications. DASFAA 2016, vol 9642. Springer, Cham.

[10] A. Saxena, K. Goebel, D. Simon and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," In International Conference on Prognostics and Health Management, Denver, CO, USA, 2008, pp. 1-9.

[11] J. Cohen, X. Huan, and J. Ni, "Fault prognosis of turbofan engines: Eventual failure prediction and remaining useful life estimation," International Journal of Prognostics and Health Management, vol. 14, 2023.

[12] Z. Lai, M. Liu, Y. Pan, and D. Chen, "Multi-dimensional self-attention based approach for remaining useful life estimation," ArXiv, vol.abs/2212.05772, 2022. unpublished.